角色融合下的一站式产研平台实践

直面角色融合中的经验差异与环境难题，用一站式产研平台让小团队办成大事

演讲嘉宾：孟小萱



自我介绍

百度 – 移动生态事业群组 – AI创新平台研发

全栈Agent工程师

主力建设 Flower – 一站式产研全链路提效平台

10+ 业务线AI提效落地经验

目录

CONTENT

01 一些背景

02 破局方向

03 融合困局

04 生产底座

05 实践故事

06 生产方式变化

01 一些背景

开发环节提效明显，全局交付提升有限

AI 代码生成率



80%

需求交付提效



20%+

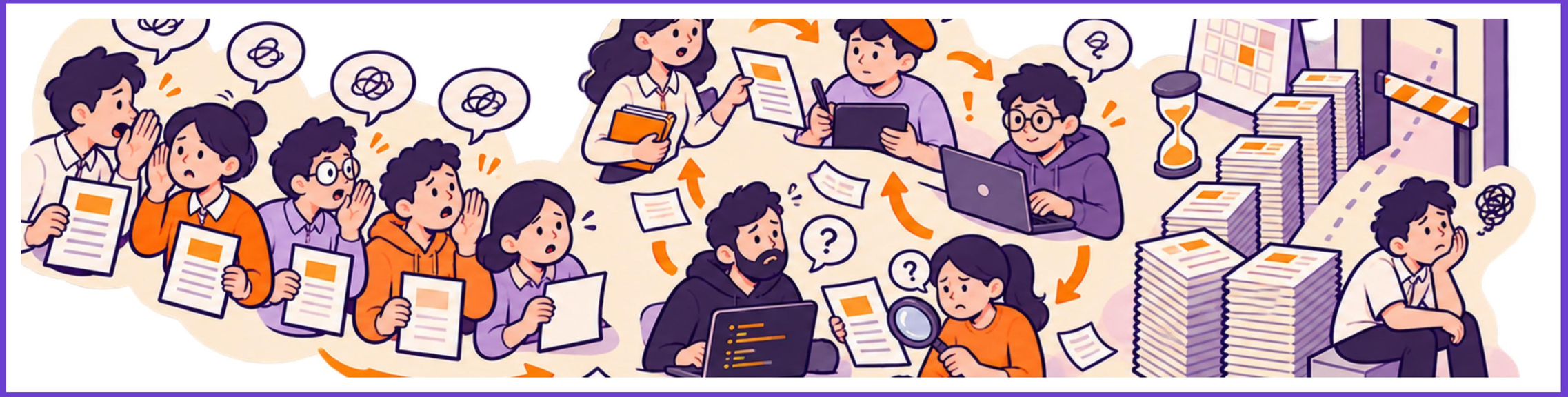
写代码只占两成多，时间主要花在沟通、等待和返工上



18 轮沟通 · 9 天相互等待　一个需求的沟通成本

3–4 天排期错位 · +2 周错过班车　赶不上双周班车

团队慢在信息失真、频繁交接和流程阻塞



01 · 信息流转

层层转述，越传越走样

需求经过几轮口头转述，到研发手里已经变了样

02 · 频繁交接

问题在角色之间来回转

产品、设计、研发、测试之间反复确认，一个问题要转好几手

03 · 流程阻塞

卡在排期和审批上

文档越写越多，审批要排队，错过班车就再等两周

02 破局方向

更少的人，
办更大的事

从单点提效走向全局交付提效

现状

单点提效接近上限

编码已被 Agent 大幅加速

→

瓶颈

协同消耗成为主因

大部分时间花在沟通、等待和交接上

→

手段

减少沟通与等待

拓展技术栈，打破角色边界

→

目标

提升全局交付效率

看端到端的交付结果

03 融合困局

一次失败的角色融合尝试



01 产设做出原型

能演示，业务方向得到验证

02 交给工程侧

期待接入现有生产系统

03 发布受阻

代码、架构与验证门禁没过

04 返工与争执

预期落空，双方重新补齐工程工作

返工只是表象，消耗的是协同与信任

大家都很努力，缺的是生产条件

表象　被迫重做　反复返工　但所有人都在努力

根因

缺失 01

共同上下文

产品和工程环境隔离，其他角色不了解工程规范

缺失 02

可执行环境

代码能编译，但在正式工程环境里跑不起来

缺失 03

可验证的质量机制

对“什么才算对”没有共同标准

AI 可以跨越角色边界，但不能越过缺失的生产条件。

04 生产底座

能跨栈做出功能，离直接交付还差一步

Agent 已具备

跨栈做出功能

+

仍然缺少

真实业务生产条件

可信的业务上下文 · 可运行的真实环境 · 可判定的验证机制

=

目标

直接交付

少了中间这一项，结果只能停在“能演示”

Flower：一站式全链路交付平台



01
云化环境
一次配置，处处运行

02
多库协同
多个代码库汇聚同一环境

03
信息共享
全流程在同一空间执行

04
质量左移
从需求阶段介入

05
上通下达
连接上下游

06
长程执行
开发、验证、修复自动循环

技术栈融合：“能写”不等于“能交付”

能写

阻碍 01

复杂环境

涉及多个库和系统，很难备好能跑起来的环境

阻碍 02

技术陌生

拿到非自己领域的代码，不知道该看哪里、按什么规范改

阻碍 03

代码掌控力

控制每一行代码已然不现实，质量风险上升

能交付

人在不熟悉的领域往往举步维艰

环境：云化 & 工程知识建设

技术栈融合 · 解决方案一

环境全靠日积月累
新人前三天光配环境

All In One 沙盒提供完整且可定制环境

多技术栈、多代码库汇聚同一环境，一次配置处处运行，不再逐人逐项搭建。

细节全在人脑里
提的代码打回又打回

让 Agent 高效操作环境

工程规范、目录结构、启动与调试方式沉淀为知识，Agent 知道去哪儿看、按什么规范改。

代码一看好像对
编译过了跑不起来

真正跑起来，而不只是编译通过

沙盒连通 IDC 网络与上下游，面向真实运行时验证。

验证：打通 Loop & 结果导向

技术栈融合 · 解决方案二

3分钟1000行代码
没人看得懂是不是能跑

以运行结果验证，而非功能演示

验收看真实链路上的结果，而不是本地能不能演示出来。

开发代码3分钟
人去验证折腾3小时

开发 — 验证 — 修复

在环境的基础上叠加长程 Loop，Agent 反复自证，直到结果正确再交出来。

代码又多又复杂
跟不上了线上质量崩溃

开发者关注结果，做好自测

人始终是需求的第一负责人，送测标准不因“代码由 Agent 产出”而下降。

产设研融合：跨角色协作容易失控



左移：提前澄清 & 风险消除

产设研融合 · 解决方案一

需求不行
返工延期问题一大堆

Agent 协助把需求写清楚

几轮对话补齐边界与场景，产出结构完整的需求；人只管表达与拍板。

基于真实代码做原型
避免上下文污染

基于真实代码生成可交互界面

原型直接在线上工程上生成，产设看到的就是之后会跑起来的样子。

细节随便写写
真开发测试吃大亏

让需求过得了“评审”

由 Agent 独立审视需求和界面，结合线上业务现状，澄清并完善。

接力：工程底座 & 协同工作

产设研融合 · 解决方案二

拉不到代码库
拿出来的页面上不了线

统一环境天然具备工程前提

代码库、工程规范、Harness 与知识已经就位，不会做出能看但不可发布的产物。

Agent说做的很好
放真实环境里只剩白屏

在真实环境里体验与验收

系统在沙盒中跑起来，产品和设计直接上手体验，按真实结果验收。

技术难题看不懂
得抱着电脑四处找人

遇到困难，请专家进现场

专家通过共享会话进入任务，直接分析、给方案，不用搬环境到别处。

底座能力 + 业务建设，“能写”才能变成“能交付”

底座能力		业务建设		效果结果
沙盒环境 知识托管 远端浏览器 高质量需求生产 上下文共享 任务协同接力	+	环境定制 知识建设 优秀工程底座 验证 Loop	→	结果导向的高质量产出 一人承担产研多角色 全链路端到端交付

05 实践故事

一项看似简单的需求

需求原文

“要上一个活动入口”

看似一句话需求

一句话需求里，藏着大量细节

需求 要上一个活动入口

↓

开发前 · 隐藏细节

需求里没写，但必须做

三端同步 | 数据防刷 | 游客不展示 | 登录态同步

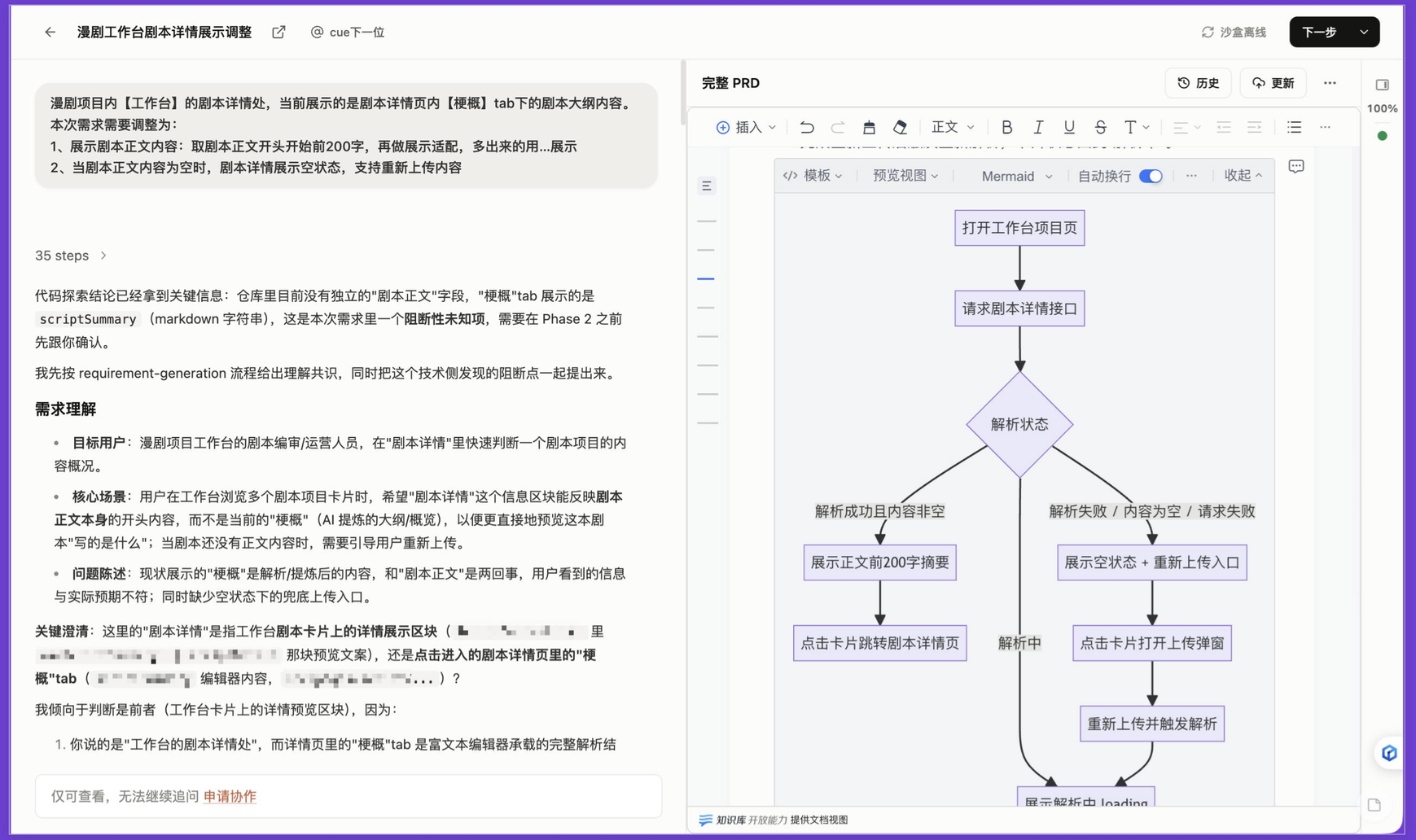
↓

测试时 · 新增边界

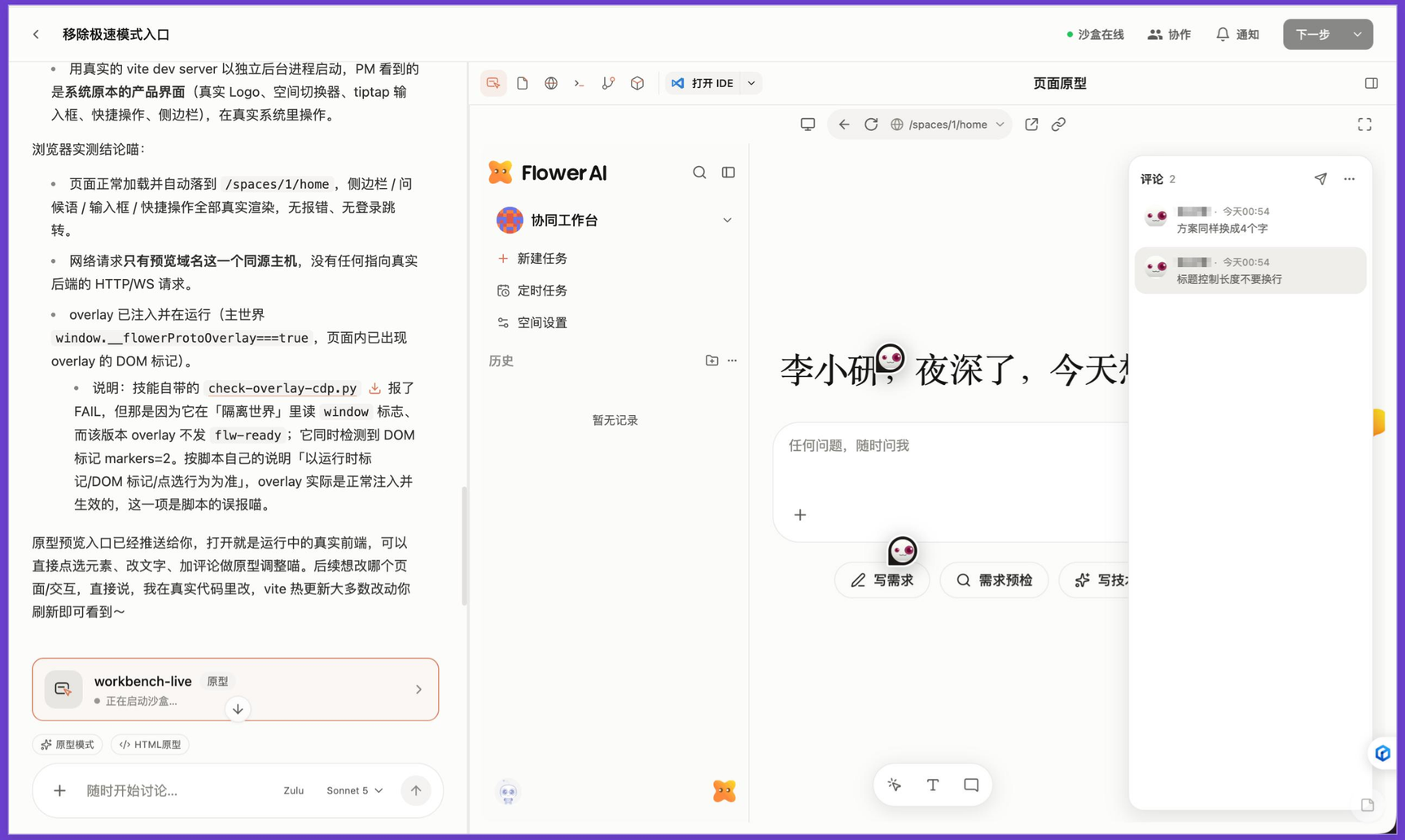
跑到测试环节才冒出来

重复操作 | 断网重试 | 记录回查

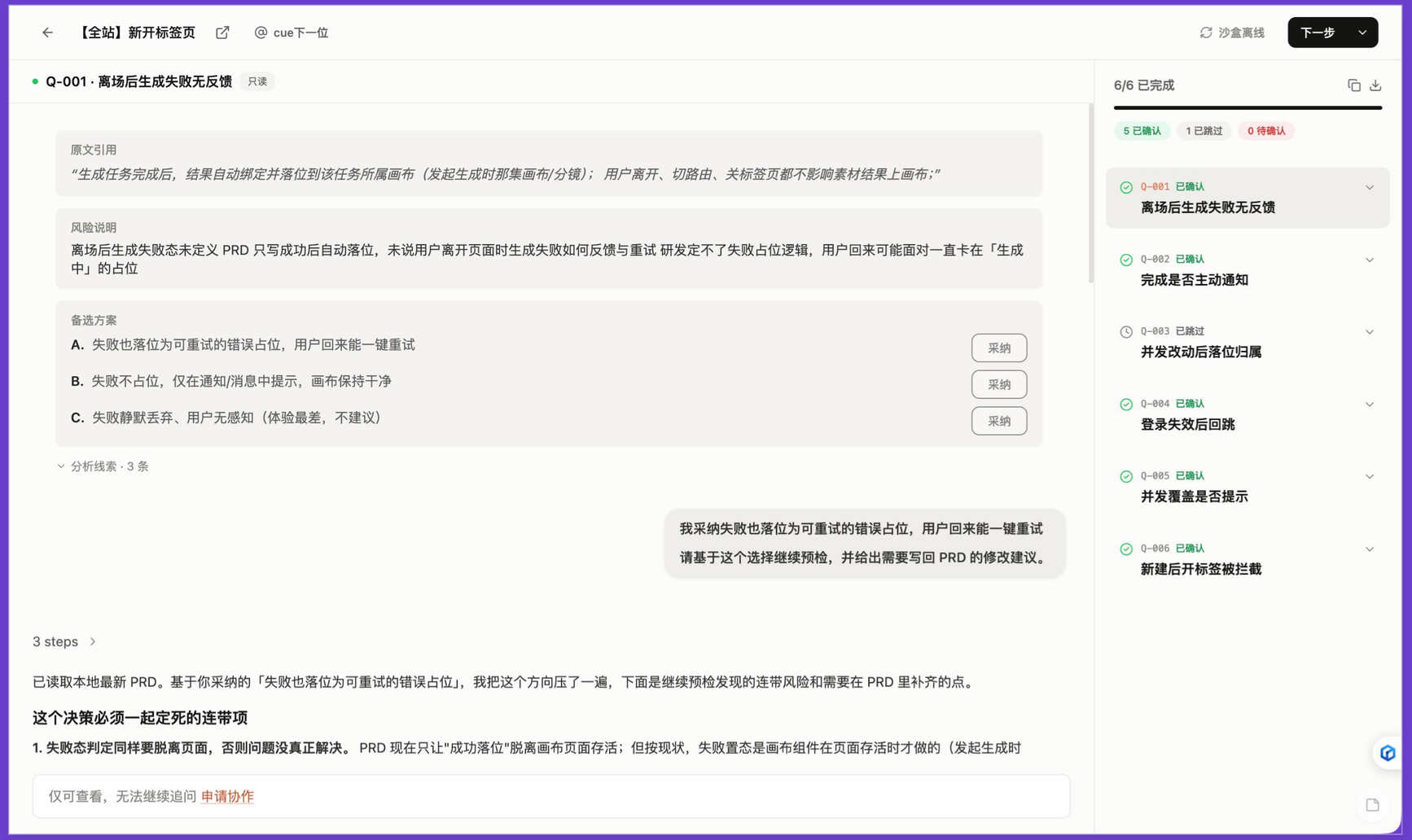
需求生成：在对话中补齐边界和场景



原型模式：真实工程代码上制作可交互原型



需求预检：对照真实代码逐条澄清问题



技术方案：定位相关代码，列出待确认项

讨论HTML技术方案复用Markdown评论与修正功能　@ cue下一位　沙盒在线　去编码

技术方案已更新。先说一个容易被忽略的前提：技术方案节点的评论 + refix **并不是完全没有**——Markdown 视图早就有整套（划词评论、抽屉、连线、选中评论触发 refix）。真正缺的是**把这套能力延伸到 HTML iframe 视图**，`TechnicalPlanDocumentPanel.tsx:129` 现在还显式写着"评论仅在 Markdown 视图可用"。所以这件事的本质是"复用现有评论体系去适配 iframe"，不是从零搭。

下面是复用判断，也是我想和你对齐的重点：

能直接复用（几乎零改动）

- 整套后端：`DocumentCommentThread` 锚定在节点主键上，Markdown/HTML 是同一个节点，评论线程天然共享，不用改表。
- 线程面板、回复、@、解决、如流通知——与视图无关，原样用。
- refix 全链路：选中评论 → 拼进 `TechnicalPlanComposer` 发送 → 同一个 Agent。因为 `htmlEnabled` 已经每轮透传，Agent 会同时重写 Markdown 和 HTML 镜像，**refix 行为在 HTML 下完全一致，不用动。**
- 锚点纯函数（`extractAnchorFromSelection` / `locateAnchor` / `applyThreadHighlights`）不绑定具体 document，可跨到 iframe 用。

需改造才能复用（iframe 适配层） 现有 `CommentAnchorPopover` / `SelectionBubble` / 连线层都假设"评论层和正文在同一个 document、同一套坐标、同一个 `getSelection()`"。iframe 打破了这三点，需要新写一层薄适配：iframe 内选区监听、rect 叠加 iframe 偏移做浮层定位、往 iframe 注入高亮样式并在每次探活重载后重放。

无法保持完全一致的点

- 最本质：一条评论的锚点是采在**某一种形态的扁平文本**上的。Markdown 和 HTML 是同一内容的两种序列化，offset/前后文都不

仅可查看，无法继续追问 申请协作

技术方案　Markdown　HTML　变更对比　发布到知识库

技术方案
- 背景与现状
- 目标
- Markdown 实现拆解
- 复用矩阵
- 一致性抉择
- 实施步骤
- 复用清单详情 >

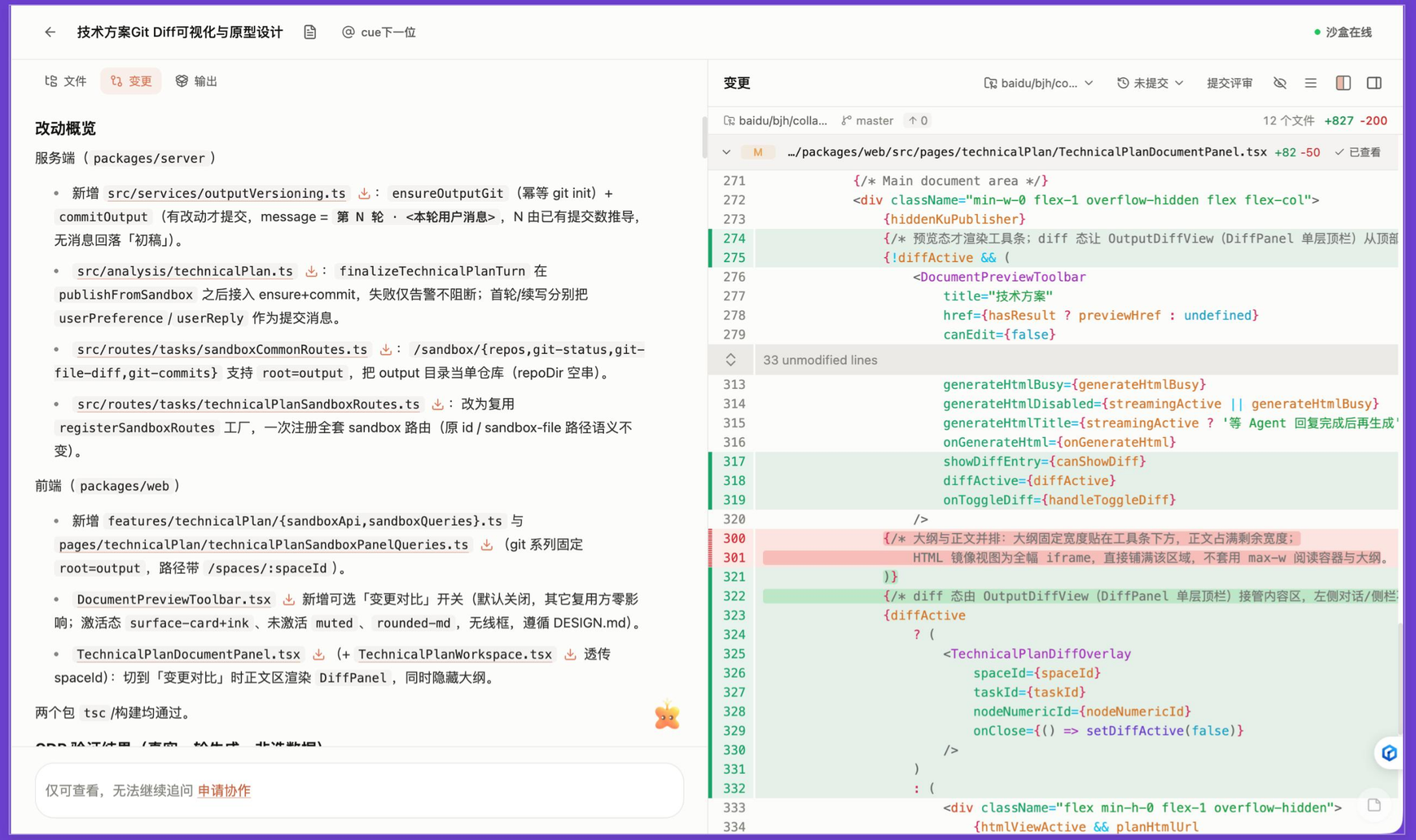
四个分层

- **数据与网络层**：`api / queries / mutations`；后端 `DocumentCommentThread`（`schema.prisma:1595`），`target_id` = 节点主键，`parent_id` 承载回复。
- **锚点层**：`extractAnchorFromSelection` 采 `anchorText + prefix + suffix + offset`；`locateAnchor` 归一化模糊匹配；`applyThreadHighlights` 用 `range.surroundContents` 注入高亮。
- **交互装配层**：`CommentAnchorPopover` / `CommentThreadPanelSuspense` / `CommentConnectorOverlay` —— 均假设评论层与正文同 document、同坐标、同一个 getSelection()。
- **refix 流**：选中评论 → `buildSelectedCommentsPrompt` 追加进 Composer → 同一 Agent（`htmlEnabled` 每轮透传）→ 完成后 `resolveComments` 置 RESOLVED。

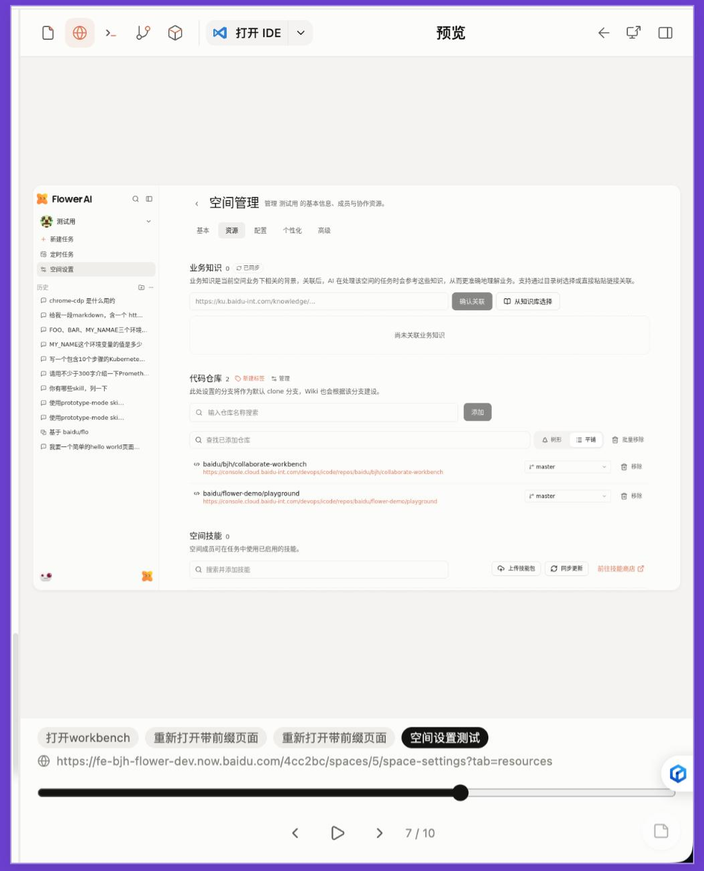


Mermaid 图元锚点是"非纯文本锚点"的先例：不新增数据库字段，把结构化锚点编码进 `anchorPrefix`（`mermaid:v1|…`），高亮改为在 SVG 上打属性标记。这是后续方案 B 可复用的关键范式。

编码实现：变更看得见，结果可验证



运行验证：在真实环境中逐条跑 Case



访问地址

http://fe-bjh-flower-dev.now.baidu.com/94bf3a/（server:9001 / web:9002，均在线）

Caselist 与截图（真实操作，节点 39「故事开放平台整合」，登录用户 孟小萱）

Case 1 — Markdown 预览 +「生成 HTML」入口（htmlEnabled=false）：工具栏右侧仅一个描边小按钮「生成 HTML」，正文为 Markdown。



Case 2 — 点击「生成 HTML」→ 两步式生效：toast「已开启 HTML 富媒体镜像，正在生成…」，「生成 HTML」按钮折叠消失、原地出现 Markdown|HTML 分段切换，同时触发一轮补齐（对话进入流式）。

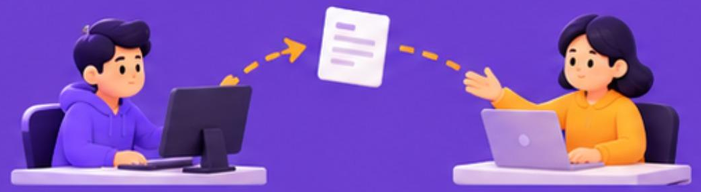


结果总览

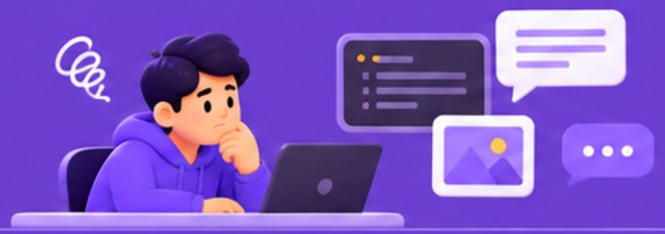
入口	conversationId	选定 effort	落库	详情回读(刷新/服务端读取)	传入 zulu
chats 对话	89be338e...	low	✅	✅ low	✅ `--effort low`
极速模式 easy_task	2465f40e...	max	✅	✅ max	✅ `--effort max`
需求生成	9eadf9c8...	low	✅	✅ low	✅ `--effort low`
技术方案	4fb209ac...	extra_high	✅	✅ extra_high	✅ `--effort extra_high`
代码生成	6a9c60ee...	max	✅	✅ max	✅ `--effort max`
需求预检	d7367ca8...	medium	✅	✅ medium	✅ `--effort medium`

接力协作：把人拉进现场，不用再把现场搬给别人

	以前 · 各自隔离	现在 · 同一现场
产出交接	工作环境彼此隔离，产物只能转交	前一棒的产出直接成为下一棒的上下文
遇到问题	截图、贴日志、重述背景	专家进入任务现场，直接分析并给出方案
专家投入	临时 Review 时重新补背景	需求、方案、环境和代码都在一处













交付结果

传统模式 | 2 人周

本次实践 | 3 人天

等待与交接

产品 → 前端 → 服务端 → 测试，多次排期

→ 同一任务现场连续推进

联调与返工

环境不齐，联调后才暴露问题

→ 真实环境中持续验证

交付产物

代码 + 口头说明

→ 代码提交 · 验证产物 · 线上效果

06 生产方式变化

交付链路最后只看两件事

01 · 产品侧

需求有没有想清楚

02 · 研发侧

实现有没有做对

业务工程的变化：编码向 Harness 建设转移

过去

业务工程 ≈ 编码

现在 | 业务负责把生产条件补齐

01



环境

可重复启动的运行环境

02



知识

网络拓扑、环境配置、上下游依赖

03



Harness

验证要求、架构约束、Review 规则

04



协作方式

任务 Owner、Harness Owner、专家

底座支撑：Flower 一站式平台 | Workspace · 沙盒 · 工具 · 协作

研发效率度量的变革：从看过程产出，到看交付结果

过去 · 看 Agent 做了多少

~~代码生成速度~~

~~Agent 使用次数~~

现在 · 看需求交付得怎样

↓ 越短越好

端到端交付时长

↓ 越少越好

人工投入

↓ 越少越好

交接等待

↓ 越少越好

返工次数

核心指标 ↑

交付到用户的吞吐

一定时间内真正交付到用户手中的需求量

业务能力的三个建设阶段

阶段 01

搬上来

把代码、知识和环境放进 Workspace

阶段 02

跑起来

用真实任务和 Harness 跑通端到端交付

阶段 03

留下来

把验证过的经验和能力留下来复用

Agent 每次都会失忆，经验要留在 Harness 里

同一个坑，每个任务都要重新踩一遍

场景 01

环境架设步骤很长

几十轮试错才跑起来，下个任务照旧

场景 02

文档与事实不符

接口早改了、目录早搬了，Agent 照着旧文档走偏

场景 03

代码检索效率低

几十个代码库里找一个模块，找入口比改代码还慢

场景 04

人工反复介入修正

这里不对、那里要注意，口头纠偏只存在于这一次会话

→

能积累的是 Harness

把失败转成资产

持续调整整个Harness，脚本、镜像、知识、规则、验证 Case 留下来，下一次任务用得上、效果好。

将会话视为知识，Agent自我进化

WHAT

质量

是否通过验收标准与回归 Case，做对是前提

成本

模型轮次、Token、端到端耗时，暴露试错浪费

人工介入

被打断几次、专家答疑几次，依赖是否下降

HOW

01
会话托管
任务在线执行，自动记录任务过程

02
分析
回看缺陷、消耗、人工介入等特征

03
聚类
批量归并，分清偶发与共性

04
提案
Agent 给出具体修改，附证据

05
人把关
判断是不是真问题

06
放行后优化
Agent 落进 Harness，回测验证

自进化：Harness 持续改进的闭环



示例：从五十次调用收敛到两步

之前 · 模型调用 **50+** 大量重复与试错

01 读取 README
02 查看 package.json
03 理解包结构
04 检查 Node 版本
05 切换 Node 版本
06 安装依赖
07 安装失败
08 清理缓存
09 安装依赖
10 切换 Node 版本
11 安装依赖
12 查找启动脚本
13 阅读启动脚本
14 研究环境变量
15 查找 .env 示例
16 设置变量
17 启动服务
18 启动报错
19 查看日志
20 研究环境变量
21 设置变量
22 启动服务
23 端口占用
24 查找占用进程
25 结束进程
26 启动服务
27 检测服务可通
28 下游连接失败
29 查找下游地址
30 配置代理
31 设置变量
32 启动服务
33 检测服务可通
34 接口 404
35 研究路由配置
36 修改路由
37 启动服务
38 检测服务可通
39 进程退出
40 查看日志
41 配置进程守护
42 启动服务
43 检测服务可通
44 注册环境
45 查找访问地址
46 修改路由
47 检测服务可通
48 页面白屏
49 重新构建
50 检测服务可通

重复操作：启动服务 ×6、检测服务可通 ×6、设置变量 ×3、安装依赖 ×3　　报错后重试

→

现在 · 脚本步骤 **2**

STEP 01

准备环境

一个脚本准备全部前置环境：Node 版本、依赖安装一次到位

STEP 02

启动服务

一个脚本完成服务启动、环境注册、服务启动、进程管理、网关配置，直接得到可访问地址

任务在跑，Harness 也在跑



THANKS

孟小萱